

**THE UNIVERSITY OF GHANA**



**UNIVERSITY OF GHANA**

**NAME: ASARE ADOM KWAKU**

**INDEX NUMBER: 22304024**

**TEAM NAME: SThrive**

**PROJECT TITLE: SThrive Network**

**PROGRAMME: COMPUTER SCIENCE**

**COURSE: SOFTWARE ENGINEERING (DCIT 208)**

**LECTURER: PROFESSOR MARK ATTA-MENSAH**

**INDIVIDUAL GITHUB PORTFOLIO: <https://github.com/Adom-Asare>**

**GROUP GITHUB PORTFOLIO: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrive/tree/main>**

**DATE: 09-07-26**

## **USE CASE**

**Use Case ID:** UC12

**Use Case Name:** RSVP to Free Event

**Primary Actor:** Free Member / Pro/Elite Member

**Secondary Actors:** Supabase (PostgreSQL, atomic write), Email Service (Resend/SendGrid)

A logged-in member finds a free event in the event calendar and clicks RSVP. The system must atomically check whether capacity is available and either confirm the RSVP (if a slot exists) or place the member on the waitlist (if the event is full). Two members clicking RSVP at the exact same moment for the last available slot must never produce two confirmed RSVPs — the capacity check and the count increment must happen as a single, indivisible database operation. If a confirmed attendee later cancels, the first waitlisted member is automatically promoted and notified.

## **USER STORY**

**Story ID:** US14

*As a member, I want to RSVP to a free event, so that my spot is reserved and I receive event details.*

**Precondition:** Member is authenticated. Event exists with status: scheduled. Member has not already RSVPd to this event.

**Postcondition:** RSVP record exists with status: confirmed or waitlisted. confirmedCount incremented by exactly one on the confirmed path only.

## **ACCEPTANCE CRITERIA**

### **AC1 — Confirmed RSVP:**

- Given a member RSVPs to an event with available capacity, when the request is processed, then the RSVP is created with status: confirmed, the event's confirmedCount is incremented by 1, and a confirmation email is sent.

### **AC2 — Waitlist (concurrent race condition):**

- Given two members RSVP simultaneously for the last available spot, when both requests reach the database, then the atomic operation ensures exactly one is confirmed and the other is placed on the waitlist — no overbooking is possible.

**AC3 — Already RSVPd:**

- Given a member who has already RSVPd attempts to RSVP again, when the request is made, then the system returns 409 with the message 'You have already registered for this event.'

**AC4 — Waitlist promotion on cancellation:**

- Given a confirmed attendee cancels their RSVP, when the cancellation is processed, then the first waitlisted member is automatically promoted to confirmed and receives a notification email.

**AC5 — Email failure fallback:**

- Given the email service is unavailable at the time of RSVP confirmation, when the RSVP is saved, then the RSVP record is still created successfully and the email is queued for retry — the user is not shown an error.

**NON-FUNCTIONAL REQUIREMENT**

**NFR ID:** NFR-P02

**Category:** Performance — Correctness Under Concurrency

The RSVP capacity check must be implemented as a single atomic database operation — the capacity check and the counter increment must occur together as one indivisible unit. The naive 'check-then-write' pattern (read confirmedCount, check against capacity, then increment in separate steps) is explicitly prohibited, as it produces a race condition under concurrent load: two requests can both pass the capacity check before either has written, resulting in overbooking.

*The implementation must use Supabase's PostgreSQL transaction (or equivalent atomic UPDATE with a WHERE condition) to guarantee that the capacity threshold is enforced at the database level, not only the application level. This requirement applies regardless of how many server instances are running simultaneously.*

## **TEST IDEA**

**Test Name:** Concurrent RSVP Race Condition — Last Slot

**Test Type:** Concurrent Integration Test

**Mapped to:** AC2, NFR-P02

### **Setup:**

- Create a test event in the database with capacity: 1 and confirmedCount: 0.
- Create two distinct authenticated test users, User A and User B.

### **Execution:**

- Fire two simultaneous POST /api/events/:id/rsvp requests — one from User A and one from User B — at the same millisecond using Promise.all() or equivalent.

### **Expected result:**

- Exactly one RSVP record has status: confirmed.
- Exactly one RSVP record has status: waitlisted.
- The event's confirmedCount equals 1, not 2.
- No duplicate confirmed RSVPs exist in the RSVP table for this event.

### **Why this test matters:**

Without an atomic database operation, this test will intermittently produce two confirmed RSVPs — catching the race condition requires simulating real concurrency, not sequential calls. If the atomic implementation is correct, this test passes 100% of the time regardless of timing.

## **REQUEST FLOW EXPLANATION**

Feature: RSVP to Free Event (UC12) | Platform: SThrive Network

### **Step 1 — User and device:**

A logged-in member is using any modern browser (desktop or mobile) on the SThrive Network platform. She is viewing the event calendar page and clicks the RSVP button on a free event.

### **Step 2 — Browser / Frontend (Next.js):**

The React (Next.js) frontend sends an HTTPS POST request to the API route. The request includes the member's JWT access token in the Authorization header, and the event ID in the URL path. No sensitive body data is required for a free event RSVP — the member's identity comes from the JWT, not a request body field.

### **Step 3 — Domain / DNS:**

The browser resolves the platform domain (e.g. shethrivesnetwork.org) to an IP address via DNS and routes to Vercel, which hosts both the Next.js frontend and the Next.js API routes. The team currently plans to use a custom domain connected to Vercel; DNS routing is handled automatically by the Vercel platform.

### **Step 4 — Protocol:**

HTTPS (TLS). All requests between client and server are encrypted. The JWT access token in the Authorization header is protected in transit — this matters because a leaked token would allow an attacker to RSVP or make requests on behalf of the member.

### **Step 5 — Web / API Server (Next.js API Route on Vercel):**

The Next.js API route at POST /api/events/[id]/rsvp receives the request. Middleware runs in sequence: first, the JWT token is verified (if invalid or expired, a 401 is returned immediately). Then, Upstash Redis rate limiting checks how many RSVP requests this IP/user has made recently.

### **Step 6 — Application Logic (critical — atomic operation):**

The route handler executes an atomic database transaction via Supabase:

```
UPDATE events
SET confirmed_count = confirmed_count + 1
WHERE id = :eventId AND confirmed_count < capacity
RETURNING *;
```

If this UPDATE returns a row (the condition was true and the increment succeeded), a confirmed RSVP is created. If it returns nothing (confirmed\_count has already reached capacity), a waitlisted RSVP is created instead. Because both the check and the increment

happen in a single SQL statement, no two requests can both pass the capacity check simultaneously — the database enforces the constraint at the row level.

### **Step 7 — Database / External Service (Supabase / PostgreSQL):**

The Supabase PostgreSQL database processes the atomic UPDATE and the subsequent INSERT into the rsvps table. A unique compound index on (event\_id, user\_id) provides a second layer of protection against duplicate RSVPs from the same user.

### **Step 8 — Response:**

On the confirmed path: 201 Created with JSON body { success: true, status: 'confirmed', rsvpId }. On the waitlisted path: 200 OK with { success: true, status: 'waitlisted', rsvpId }. On duplicate RSVP: 409 Conflict. On database failure: 503 Service Unavailable (server does not hang — it fails fast and clearly).

### **Step 9 — User sees result:**

The frontend reads the response status field and shows either 'You are registered for this event!' (confirmed) or 'You have been added to the waitlist' (waitlisted). The RSVP button is replaced with the appropriate status label. A confirmation email is triggered asynchronously — the user receives this separately; the UI response does not wait for it.

### **Failure point:**

The most likely failure point is the Supabase connection dropping mid-transaction (similar to the Atlas SRV connection issues encountered during the ThinkBoard project). If this happens, Supabase rolls back the transaction automatically — the RSVP count is never partially incremented. The API returns 503, and the frontend prompts the user to try again.

### **Security / privacy concern:**

The member's identity is verified via JWT before any database operation is attempted — the RSVP endpoint is not publicly accessible. Unauthenticated requests are rejected at the middleware layer with 401 before any database query runs. The event ID is validated to prevent RSVPs to non-existent or cancelled events.

## **AI DISCLOSURE**

*I used AI assistance (Claude, Anthropic) in preparing this submission. The tool was used to help structure the traceability card format, verify technical accuracy of the atomic SQL operation and request-flow steps, and ensure the content aligns with the team's SRS (D2) and architecture documentation. The use case, user story, acceptance criteria, NFR, and test idea are drawn directly from the team's own deliverables (D2 SRS) and client meeting evidence. I have reviewed and can explain and defend every part of this submission.*

**Tool:** Claude (Anthropic)

**Prompt Summary:** Requested a complete IA3 traceability card for the UC12 RSVP flow.

**How AI Was Used:** Claude was used to generate an initial structure for the traceability card, including technical wording, a sample SQL statement, and the request-flow steps. These suggestions served as a starting point for drafting the document.

**What Was Rejected or Changed:** The AI-generated content was not copied verbatim. I reviewed every section, rewrote the text where necessary, and made revisions to ensure it accurately reflected my understanding and met the assignment requirements.

**How It Was Verified:** The final version was verified by comparing it with the D2 Software Requirements Specification (SRS), the D3 Product Backlog, and the team's architecture documentation to ensure consistency, correctness, and alignment with the project.